

1-Task Title: Bezout's Co-efficient:

Code:

Bezouts_coefficient.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define newl cout<<endl

int gcd;
vector<int>quotient_gcd(int a,int b, vector<int>q)
{
    if(b==0)
    {
        gcd=a;
        return q;
    }
    if((a/b)!=0)
    {
        q.push_back(a/b);
    }
    return quotient_gcd(b,a%b,q);
}

map<int,int>Bezouts_coefficient(int a,int b)
{
    vector<int>s={1,0},t={0,1},q;
    q=quotient_gcd(a,b,q);
    for(int i=2;i<=q.size();i++)
    {
        s.push_back(s[i-2]-q[i-2]*s[i-1]);
        t.push_back(t[i-2]-q[i-2]*t[i-1]);
    }
    map<int,int>res;
    res[max(a,b)]=s.back();
    res[min(a,b)]=t.back();
    return res;
}
```

```
#include"Bezouts_coefficient.cpp"
int main()
{
    cout<<"Enter 2 numbers-> ";
    int a,b;
    cin>>a>>b;
    map<int,int>res=Bezouts_coefficient(a,b);
    newl;
    cout<<"Bezouts Coefficient of "<<a<<" is "<<res[a];
    newl;
    cout<<"Bezouts Coefficient of "<<b<<" is "<<res[b];
    newl;
}
```

Output:

```
Enter 2 numbers-> 252 198
```

```
Bezouts Coefficient of 252 is 4
```

```
Bezouts Coefficient of 198 is -5
```

```
PS F:\RUET CSE LAB\Discrete Mathematics 2102\8. 20-04-2024>
```

Discussion:

Here I have used Back Extended Euclidean Algorithm to calculate the Bezout's Coefficients. When the GCD of 2 values is 1 then the Bezout's Coefficients are the inverse value to each other's. These values are necessary for calculating the simultaneous congruence relation like Chinese Remainder Theorem.

2-Task Title: Chinese Remainder Theorem:**Code:**

```
#include"Bezouts_coefficient.cpp"
int main()
{
    cout<<"Enter Total Congruence Relations (a,m): ";
    int n;
    cin>>n;
    ll M=1;
    int x=0;
    vector<pair<int,int>>>CR;
    for(int i=1;i<=n;i++)
    {
        int a,m;
        cin>>a>>m;
        CR.push_back({a,m});
        M*=m;
    }
    for(int i=0;i<n;i++)
    {
        ll mk=M/CR[i].second;
        map<int,int>inverse=Bezouts_coefficient(CR[i].second,mk);
        x+=(CR[i].first*mk*inverse[mk]);
    }
    if(x%M<0)
    {
        x=(x%M)+M;
    }
    cout<<"The value of x is ";
    cout<<(x%M);
}
```

Output:

```
Enter Total Congruence Relations (a,m): 3
7 9
0 4
4 5
The value of x is 124
PS F:\RUET CSE LAB\Discrete Mathematics 2102\8. 20-04-2024>
```

Discussion:

Chinese Remainder Theorem is a process to calculate the simultaneous congruence relation where the moduli are pairwise relatively prime. Here I have used Bezout's theorem to calculate the inverse. After that I put them in the equation of the theorem to calculate the minimum positive number which satisfies the congruence relations.